

Algorithmic Complexity

Course notes - Department of Computer Science

1. Why complexity matters

Measuring an algorithm by the wall-clock time it takes on one machine tells you about the machine as much as the algorithm. Complexity analysis instead counts how the number of elementary operations grows as the input grows, which is a property of the algorithm alone and survives a change of hardware, language or compiler.

We care about growth because constant factors stop mattering once inputs get large. An algorithm running in $100n$ steps beats one running in n^2 steps for every n above 100, and the gap widens without bound.

2. Asymptotic notation

Big-O gives an upper bound: $f(n) = O(g(n))$ when there exist constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Big-Omega gives the corresponding lower bound, and Big-Theta applies when the upper and lower bounds agree, pinning the growth rate exactly.

In practice we quote the tightest bound we can prove. Saying that binary search is $O(n)$ is true but useless; it is $\Theta(\log n)$.

3. Common growth rates

Constant time $O(1)$ covers array indexing and hash table lookup in the average case. Logarithmic time $O(\log n)$ arises whenever the input is halved each step, as in binary search or balanced tree descent. Linear time $O(n)$ is the cost of examining every element once.

Linearithmic time $O(n \log n)$ is the bound for comparison-based sorting, achieved by merge sort and heap sort. Quadratic time $O(n^2)$ appears in nested loops over the same collection, and exponential time $O(2^n)$ in exhaustive search over subsets.

4. Analysing recursive algorithms

Recursive algorithms are described by recurrence relations. Merge sort splits its input in half and does linear work merging, giving $T(n) = 2T(n/2) + O(n)$, which resolves to $\Theta(n \log n)$.

The master theorem handles recurrences of the form $T(n) = aT(n/b) + f(n)$ by comparing $f(n)$ against $n^{\log_b a}$. Whichever term dominates determines the result; when they match, a logarithmic factor is introduced.

5. Worst, average and amortised cost

Worst-case analysis bounds every input and is what we usually quote. Average-case analysis assumes a distribution over inputs, which is why quicksort is described as $O(n \log n)$ on average despite a quadratic worst case.

Amortised analysis averages cost over a sequence of operations. A dynamic array that doubles its capacity when full performs an expensive copy occasionally, but the cost spread across all insertions is constant per insertion.